IaCSecBench: A Reproducible Benchmark and Finding-Normalization Methodology for Evaluating Infrastructure-as-Code Security Validation

Manideep Chittineni

Abstract—Infrastructure as Code (IaC) has become the dominant mechanism for provisioning cloud platforms, yet pre-deployment security validation remains fragmented across static scanners, policy engines, and infrastructure testing frameworks. Comparative evaluation of these tools is impeded by two problems: candidate tools operate at incompatible abstraction levels, from lexical scanning of source templates to policy evaluation over resolved execution plans, and each reports findings in a bespoke schema. Benchmarks authored by tool developers compound the difficulty, since the same interpretation of a security control tends to define both the policy and the ground truth. This paper presents IaCSecBench, an open framework for evaluating layered IaC security validation pipelines, together with a finding-normalization methodology that maps heterogeneous scanner output onto a canonical control taxonomy and thereby permits confusion-matrix comparison across tools with unrelated report formats. The framework composes repository-edge data validation, native Terraform module testing, and policy evaluation over compiled plan documents. Over a labelled corpus we evaluate two independent third-party scanners against the pipeline's own layers and their union, and assess generalization separately on independently sourced configurations. The reference pipeline does not lead the comparison, and we report that outcome as measured. Detection effectiveness is reported with exact Clopper–Pearson intervals, and pairwise differences with exact binomial tests under family-wise correction. We further report a mechanically enforced corpus admissibility procedure that rejects cases which cannot support a defensible ground-truth label. The corpus, normalization engine, execution harness, and analysis code are released as a replication package.

Index Terms—Cloud computing security, continuous integration, DevSecOps, Infrastructure as Code, policy as code, software benchmarking, static analysis.

I. Introduction

CLOUD platforms are increasingly provisioned through Infrastructure as Code (IaC), in which infrastructure topology is expressed declaratively in version-controlled artefacts [1], [2], atop service models whose elasticity makes such automation a practical necessity [3]. The property that makes IaC attractive—deterministic, repeatable application of a declared state—also amplifies its failure modes: a single insecure declaration, such as an unrestricted ingress rule or a disabled encryption setting, propagates automatically to every environment that consumes the module. Empirical studies of configuration repositories confirm that such defects are common rather than exceptional, recurring as hard-coded secrets, excessive privileges, and insecure defaults [4], [5], [6].

Because remediation after provisioning necessarily implies a window of exposure, contemporary practice shifts validation earlier, into continuous integration [7], [8]. Doing so raises an evaluation problem the literature has not resolved. Candidate gates operate at incomparable abstraction levels: lexical scanning of HCL source, abstract-syntax tree analysis, and policy evaluation over resolved plan state. Each reports findings in a bespoke schema, with rule identifiers that carry no shared semantics. Comparative claims in this space are consequently difficult to reproduce, and when a benchmark is authored by the developer of one of the tools under comparison, agreement between that tool and the ground truth is close to guaranteed by construction.

This paper addresses the evaluation problem rather than proposing a further analyser. Three commitments shape the design.

First, normalization before comparison. A canonical control taxonomy mediates between native rule identifiers and benchmark ground truth, so that a finding is credited when it identifies the control a case was authored to violate, and not merely when the tool emitted something. Findings whose rule identifiers the taxonomy does not cover are counted and reported rather than discarded, because silently treating an unmapped detection as a miss biases the comparison toward whichever tool the taxonomy was authored around.

Second, admissibility is mechanically enforced. A case is admitted only if it is valid HCL2 that references resource types the declared provider defines, carries a non-contradictory ground-truth label, and resolves to an unambiguous canonical control. The reported corpus size is the admissible count, not the number of entries in a catalogue.

Third, measurement is separated from assumption. A tool that is not installed in the measurement environment is reported as such and excluded; it is never represented by an assumed detection rate. Latency is sampled repeatedly and reported with dispersion.

A. Contributions

- A finding-normalization methodology and canonical control taxonomy that render heterogeneous IaC

M. Chittineni is a Senior Platform, Cloud and DevOps Engineer, United Kingdom (e-mail: manideep.chittineni@gmail.com).

scanner output commensurable, with three explicit matching strictness levels that separate correct attribution from incidental detection.
- A corpus admissibility procedure that enforces the stated inclusion criteria mechanically, together with the measured admissibility of the released corpus.
- An execution harness that records raw tool output, resolved tool versions, and repeated latency samples, and that refuses to emit results when no case is admissible.
- A statistical protocol appropriate to paired, small-sample scanner comparison: exact Clopper–Pearson intervals [9], [10], the exact binomial form of McNemar's test [11], [12], and Holm–Bonferroni correction across the tool family [13].
- An open replication package containing the corpus, taxonomy, harness, and analysis code, in which every reported table is a generated artefact.

II. Background and Related Work

A. Security Weaknesses in Infrastructure as Code

Rahman et al. established that infrastructure scripts contain recurring security smells, first across Puppet [4] and subsequently in Ansible and Chef [5], and derived a defect taxonomy for IaC scripts [14]. Complementary work characterises misconfiguration in Kubernetes manifests [15], [16] and security practice in Terraform repositories specifically [6]. Quality and defect-prediction metrics for infrastructure code have also been catalogued [17], [18]. This body of work motivates automated pre-deployment validation but does not address how competing validation tools should be compared.

B. Static Analysis of IaC and Existing Benchmarks

Chiari et al. survey static analysis for IaC and note the absence of shared evaluation infrastructure [19]. The closest prior art to this paper is GLITCH, which detects security smells across several IaC technologies via an intermediate representation and is evaluated against a labelled oracle dataset with a tool comparison [20]. IaCSecBench differs in target and in claim: GLITCH contributes a detector with polyglot coverage, whereas this work contributes the normalization and admissibility methodology needed to compare detectors that report incommensurable findings, and applies it to plan-level as well as source-level evaluation.

Opdebeeck et al. show that control- and data-flow analysis materially changes security-smell detection in IaC [21], [22]. That result motivates the plan-level layer evaluated here: a compiled Terraform plan is a resolved artefact in which interpolation, conditionals and expansion have already been applied, so evaluating it sidesteps a class of analysis difficulty rather than solving it.

Methodologically, this paper follows the static-analyser evaluation tradition established by Habib and Pradel, who show that headline detection rates are highly sensitive to what counts as a match [23], and the benchmark-construction cautions of the OWASP Benchmark project [24]. The three-level matching criterion in §III-D is a direct response to the first.

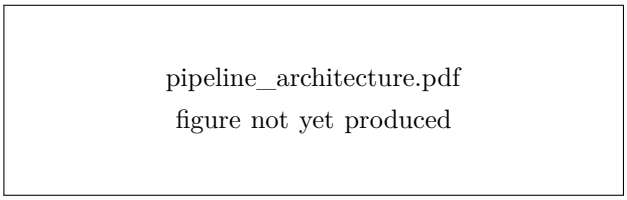

Fig. 1. Layered validation pipeline: repository edge data validation, native module testing, and policy evaluation over compiled plan documents.

C. Policy as Code and Native Infrastructure Testing

Open Policy Agent provides general-purpose policy evaluation over structured documents [25], and Terraform provides a native test harness for module behaviour [26]. Compliance requirements are drawn from the CIS AWS Foundations Benchmark [27] and NIST SP 800-53 [28]. IaCSecBench composes these mechanisms; it does not extend them.

III. Framework Architecture

IaCSecBench composes three validation layers, each addressing a distinct failure class.

A. Layer 1: Repository-Edge Data Validation

The first layer prevents sensitive material from entering version control through schema verification, detection of sensitive fields, and pattern-based identification of personal data. This layer operates on repository content rather than infrastructure semantics, and its characteristic failure mode is over-detection: high-entropy strings in output declarations are frequently flagged as credentials. Its false-positive behaviour is therefore reported separately in §VI rather than aggregated with the policy layers.

B. Layer 2: Native Module Testing

The second layer validates module behaviour using Terraform's native test framework [26], covering variable constraints, output correctness, conditional resource creation, and tagging requirements without provisioning cloud resources.

C. Layer 3: Policy Evaluation over Compiled Plans

The third layer converts a Terraform plan to its JSON representation and evaluates Rego policies against it [25]. Because the plan is produced after expression resolution, policies observe concrete attribute values rather than unevaluated references. Plan generation is configured so that provider credential validation is disabled, which permits evaluation in continuous integration without cloud account access; this is a precondition for the layer being usable at all, and is implemented by an injected provider override rather than by relaxing any policy.

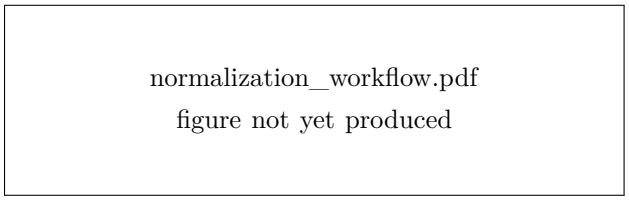

Fig. 2. Finding-normalization engine and evaluation pipeline.

D. Finding Normalization

Comparing heterogeneous scanners requires a canonical representation. A finding is normalised to

$$\mathcal{F} = \langle c,\ t,\ \kappa,\ r,\ s \rangle, \tag{1}$$

where c identifies the benchmark case, t the tool, κ the set of canonical controls the tool's native rule maps to, r the resource address, and s the reported severity. The mapping from native rule to κ is many-to-many: a coarse rule may legitimately cover several controls, and such rules are reported explicitly because they weaken attribution precision.

Detection is then evaluated at three strictness levels:

- control: some finding's κ intersects the controls the case was authored to violate. This is the primary criterion.
- resource: some finding names the resource address the ground truth identifies, irrespective of which rule fired. This isolates the correct-resource-wrong-reason case.
- any: the tool emitted any finding within the case. This criterion overstates detection and is reported only to quantify how much of a tool's apparent performance is incidental.

Reporting all three is deliberate: adopting a single permissive criterion is the most common mechanism by which scanner benchmarks overstate detection [23]. Where the ground truth does not name a resource address, the resource criterion is inapplicable and is reported as such, not as a failure.

IV. Threat Model

The evaluation scope is defined using STRIDE [29]. Protected assets are infrastructure configurations and compiled plan states; ingested datasets; policy and test definitions; and pipeline credentials. Trust boundaries separate the developer workspace from version control, the evaluation runtime from the target repository, the compiled plan from the policy engine, and client applications from cloud service endpoints.

Adversaries are modelled at three capability tiers: passive repository scanning for exposed secrets; submission of misconfigured changes containing privilege escalation or disabled encryption; and interference with pipeline execution or policy definitions.

Four domains are out of scope. Post-deployment runtime compromise, including container and hypervisor vulnerabilities, is not addressed. Supply-chain compromise of provider binaries or registry mirrors is not addressed. IaC technologies other than Terraform are not evaluated, since the plan-level layer presupposes Terraform plan JSON. Physical and social-engineering attacks are excluded. The threat model is stated to delimit scope; the pipeline was not subjected to adversarial testing, and no claim of resistance to the modelled adversaries is made.

TABLE I
STRIDE categories, mitigations, and the layer that enforces them.

Category	Mitigation	Layer
Spoofing	Authenticated API authorisers; transport encryption enforcement; prohibition of embedded credentials	1, 3
Tampering	Schema validation; pre-commit enforcement; evaluation of compiled plan documents	1, 2
Repudiation	Structured pipeline telemetry; recorded test execution; compliance tagging	2, 3
Information disclosure	Pattern-based personal-data detection; mandatory encryption at rest; storage public-access blocking	1, 3
Denial of service	Validation of web application firewall association, header handling, and request throttling	2, 3
Elevation of privilege	Policy denial of wildcard identity actions and unrestricted ingress ranges	3

V. Evaluation Methodology

A. Corpus Construction and Admissibility

The corpus separates a controlled collection, authored for this evaluation, from an external collection sourced independently. The distinction matters because the controlled collection shares an author with the policy set and therefore cannot provide independent confirmation of detection capability.

Admissibility is enforced mechanically rather than asserted. A case is admitted only if it satisfies all of the following:

1) it declares at least one resource or module block;
2) its ground-truth label is present and internally consistent, where a case declaring contradictory labels is rejected rather than resolved by precedence;
3) it resolves to exactly one interpretation in the canonical taxonomy, since a CIS section number may span several distinct controls and inference cannot choose between them;
4) terraform init and terraform validate succeed, which is the only check that detects a case referencing a resource type the provider does not define.

Table II reports the outcome of applying this procedure. The admissible count is the corpus size used throughout;

TABLE II
Corpus admissibility. The admissible count is the corpus size reported throughout; catalogue entries without configuration are not cases. Rejection reasons are determined mechanically by evaluation/corpus.py.

Corpus status	Cases
Catalogue entries declared	520
Configurations present on disk	48
Admissible	48
vulnerable	26
compliant baseline	22

catalogue entries without corresponding configuration are not cases and are excluded. We report the rejection reasons because they characterise the corpus more honestly than a headline size does.

B. Control Coverage

The canonical taxonomy defines 26 controls, of which 22 are exercised by at least one admissible case. Both numbers are stated because they answer different questions, and the larger one is not an answer to the question a reader is asking. The size of the taxonomy describes what the normalization engine can express; the exercised count describes what this evaluation measured. Reporting only the former would credit the corpus with four controls — API-gateway authorization and three Kubernetes workload controls — against which nothing was ever run.

The unexercised controls are concentrated in the domains the corpus does not reach at all: Kubernetes workloads and serverless authorization. Their absence is a coverage limitation of the corpus, not a negative result about any tool, and no tool is penalised for them, since a control with no case contributes to no confusion matrix.

Auditing the taxonomy against the installed tool catalogues found three defect classes in the mapping itself, all corrected. Seven controls named plan-level policy rules the policy set does not implement. Two named Checkov identifiers absent from the installed catalogue entirely, and one mapped a resource-requests check to a resource-limits control, which are distinct Kubernetes concepts. None of these can produce a spurious detection, because nothing emits the identifier; each instead inflates the apparent coverage of the layer it is attributed to, and in the first case that layer is the reference implementation. That is the direction of error that flatters the contribution under evaluation, which is why the audit was mechanical rather than by inspection, and why the resulting invariant — every plan-level identifier must exist in the policy source — is now enforced by a test.

Two controls lost their only rule to that correction and became detectable by no tool. A control in that state scores every case citing it as a miss for every tool simultaneously, which reads as unanimous tool failure rather than as absent coverage; both were removed, and neither was exercised, so no reported figure changes. Eight controls retain an unverified flag, each naming the single identifier behind it. All eight are tfsec long identifiers that this environment cannot corroborate: tfsec 1.28 provides no check-listing command, and its –filter-results option returns nothing for an unknown identifier and for a real but untriggered one alike. They are retained rather than deleted, because a wrong identifier is inert — nothing emits it, so it never matches — and the only possible effect of keeping it is to credit a comparator if it turns out to be real. Where the two options differ, we take the one that cannot flatter the reference.

C. Ground-Truth Labelling

Labels are derived mechanically, not by human rating. Each controlled case is emitted by a generator from a per-control specification that declares the intended violation, and the label follows from which member of the vulnerable/compliant pair is being written. The generator, the specification, and the emitted cases are all in the replication package, so any reader can check that a case carries the label its specification declares.

Because labels are a deterministic function of the specifications, rater disagreement is impossible by construction, and the threat that replaces it is specification error: a specification that misreads its originating control produces a wrong label reproduced identically across the whole pair. An agreement coefficient computed over the generator's own output would measure its determinism, not concurrence, so we do not report one.

We instead ran a blind second labelling pass against the threat that does apply. For each admissible case a review view was constructed containing only the Terraform configuration and the canonical control under test, with every comment line removed — the generator writes the expected label and its rationale into each file header — along with the annotation files and the case identifier, whose suffix names the class. Cases were presented in an order keyed on a hash of the case identifier, and the views were screened for residual label vocabulary. Each was then labelled from the configuration and the control text alone, with a reason recorded before any comparison.

Agreement is reported before reconciliation, since resolving disagreements guarantees consensus: 47 of 48 cases agreed, giving raw agreement of 97.92% and Cohen's $\kappa = 0.958$ against a chance-agreement baseline of 50.17%.

The single disagreement is instructive, and it is the kind of defect this pass exists to surface. A bucket configured with SSE-S3 (AES256) was labelled violating under a control titled lacks server-side encryption and citing CIS Amazon Web Services 2.1.1. That configuration does not lack server-side encryption, and CIS 2.1.1 is satisfied by SSE-S3; read against its stated text the case is compliant. The generator's rationale reveals the intent — encryption other than a customer-managed key — so the label encoded a requirement the control text never stated. That stricter requirement is also precisely what the reference policy set enforces, which makes this a concrete instance

of the construct-validity threat of Section VIII-A rather than a hypothetical one: the ground truth agreed with the reference implementation rather than with the control it cited.

We corrected the control text rather than the label, because requiring a customer-managed key is a defensible control and is what the corpus, the policy set, and the customer-key rules of both source-level scanners all test; relabelling instead would have left the pair testing nothing. The control is retitled to state the key requirement and records that it is stricter than the CIS control it cites. No confusion-matrix entry changed. A regression test now pins the citation in the taxonomy to the one in the generator specification so the two cannot drift apart silently.

Two properties of this pass bound what it establishes, and we state them rather than let κ speak unqualified. The second rater is an automated one, so this is not human inter-rater reliability; and its reading of the control texts derives from the same public documentation the specifications were written from, so the two raters are not fully independent and agreement is biased upward. Five cases had been inspected during earlier debugging and their agreement is not blind; excluding all five leaves 42 of 43 agreeing, with the disagreement outside that set. The full per-case labels, reasons, and these caveats are in the replication package.

Two further checks bear on specification error. The admissibility procedure of Section V-A rejects any case whose label is absent, internally contradictory, or resolvable to more than one control in the taxonomy. And the audit of the control map reported in Section V-B found three classes of mapping defect by mechanical comparison against the installed tool catalogues, which is the same style of check applied to the other half of the scoring path. Neither check can establish that a specification reads its control correctly; that remains an unmitigated threat, and independent relabelling by a second investigator is the obvious next step.

D. Research Questions

- RQ1: What detection effectiveness do the evaluated tools achieve on the admissible controlled corpus, and how precisely is that effectiveness estimated?
- RQ2: How much does the matching criterion change apparent detection?
- RQ3: What execution latency does each layer contribute in continuous integration, and what is the size of the native validation suite required to express the corresponding assertions?
- RQ4: Which layers account for which detections?
- RQ5: Does effectiveness persist on independently sourced configurations?

E. Measurement Environment and Tool Versions

Tool versions are resolved at execution time and recorded in results/run_manifest.json, together with the platform, processor, and repository commit. Versions are not restated in prose, because a hand-copied version string is the first thing to drift; the manifest is authoritative.

Tools that are not installed in the measurement environment are recorded as such and omitted from every table. No tool is assigned an assumed detection rate or an assumed latency.

F. Tool Selection

The comparison covers Checkov, tfsec, and standalone policy evaluation over plan documents. Selection was governed by two criteria: the tool must parse HCL2 or Terraform plan JSON without requiring cloud provider API access, and the set must span distinct analysis paradigms—syntax-tree analysis, lexical scanning, and policy evaluation. Trivy, KICS, Regula, and provider-native configuration services are excluded under the first criterion or as duplicating a represented paradigm.

Terrascan is excluded under the second criterion: its analysis paradigm is Rego policy evaluation, which the plan-level layer already represents. The consequence for the comparison is stated explicitly: policy evaluation is represented only over compiled plan documents, and no source-level policy engine appears in the tool set. A source-level Rego engine could differ from a plan-level one on configurations whose violations are resolved during planning, and this benchmark does not measure that difference. We make no claim about Terrascan's detection behaviour on this corpus, because no successful Terrascan execution was ever recorded against it.

VI. Results

All tables in this section are generated by evaluation/analyze.py from recorded raw tool output. They are regenerated by experiments/run_baselines.sh and are not edited by hand.

A. RQ1: Detection Effectiveness

Table III reports confusion matrices with exact Clopper–Pearson intervals. Intervals are reported for every proportion because a point estimate on a corpus of this size is compatible with a wide range of true values; a perfect point estimate in particular carries a lower confidence bound substantially below unity, and reporting the point estimate alone would misstate what has been established. Metrics that the corpus cannot support are marked as not estimable rather than printed as zero: specificity and the Matthews correlation coefficient [30] require ground-truth-compliant cases, and in their absence no false-positive behaviour is characterised at all.

Table IV reports pairwise comparisons. The discordant count b includes every case in which the reference is correct and the comparator is not, counting spurious findings as well as missed violations; restricting b to missed violations understates disagreement. The exact binomial p-value is reported in preference to the chi-square approximation, which is invalid when a discordant cell is small and

TABLE III
Detection performance on the admissible corpus ($N = 48$: 26 vulnerable, 22 compliant). Intervals are exact Clopper–Pearson at the 95% level. Entries marked n/e are not estimable from this corpus.

Tool	TP	FP	TN	FN	Rec. (%)	Rec. 95% CI	Prec. (%)	Prec. 95% CI	MCC
Checkov	23	0	22	3	88.46	[69.85, 97.55]	100.00	[85.18, 100.00]	0.882
IaCSecBench L1	1	1	21	25	3.85	[0.10, 19.64]	50.00	[1.26, 98.74]	-0.017
OPA (plan-level)	20	0	22	6	76.92	[56.35, 91.03]	100.00	[83.16, 100.00]	0.777
tfsec	22	2	20	4	84.62	[65.13, 95.64]	91.67	[73.00, 98.97]	0.753

TABLE IV
Exact McNemar comparisons against OPA (plan-level). b counts cases the reference classifies correctly and the comparator does not, counting both missed violations and spurious findings; c is the converse. p is the two-sided exact binomial value, p_{adj} its Holm–Bonferroni correction across the tool family. The odds ratio carries a Haldane–Anscombe correction so that a zero discordant cell yields a finite estimate.

Comparator	b	c	p	p_{adj}	OR	Cohen's g
Checkov	2	5	0.453	0.906	0.45	0.214
IaCSecBench L1	21	1	< 0.001	< 0.001	14.33	0.455
tfsec	3	3	1.000	1.000	1.00	0.000

TABLE V
Recall under three finding-matching criteria. Control credits a detection only when the tool's rule maps to the control the case violates; resource credits any finding on the expected resource; any credits any finding whatsoever. The spread measures attribution precision.

Tool	Control (%)	Resource (%)	Any (%)
Checkov	88.46	76.92	96.15
IaCSecBench L1	3.85	0.00	100.00
OPA (plan-level)	76.92	65.38	84.62
tfsec	84.62	73.08	92.31

is therefore not reported as a primary result. p-values are corrected across the tool family [13]; test selection follows established guidance for software-engineering experiments [31]. Odds ratios carry a Haldane–Anscombe correction, so a zero discordant cell yields a finite estimate; an unbounded ratio is a degenerate cell rather than a large effect, and Cohen's g is reported as the effect size.

B. RQ2: Sensitivity to the Matching Criterion

Table V reports recall under each criterion. The spread between the control and any columns measures how much apparent detection is attributable to findings unrelated to the seeded violation. A tool whose recall is high under the permissive criterion and substantially lower under the control criterion is emitting findings on these cases without identifying the vulnerability in question.

C. RQ3: Execution Cost and Suite Size

Table VI reports per-case latency as a mean with standard deviation over repeated executions. Single-run figures are not reported: variance on shared-tenancy runners is large enough that a one-decimal single-sample figure is not a measurement.

TABLE VI
Measured per-case execution latency over 5 repetitions. Values are mean ± standard deviation on the environment recorded in results/run_manifest.json. Each tool is measured at its own interface: the plan-level figure covers policy evaluation over an already-compiled plan document and excludes the terraform init and terraform plan invocations that produce it, which dominate that layer's wall-clock cost in continuous integration. It is therefore a lower bound, and is not comparable with the source-level figures, which have no equivalent preparation step.

Tool	Evaluation layer	Latency (ms)
Checkov	AST static analysis	2421.0 ± 154.8
IaCSecBench L1	Repository-edge scanning	0.2 ± 0.1
OPA (plan-level)	Rego over compiled plan	18.6 ± 1.0
tfsec	HCL lexical scanning	422.6 ± 4.6

On engineering effort we report a measurement of the native suite alone and withdraw the comparative claim. The validation suite comprises 11 .tftest.hcl files totalling 190 physical lines, of which 153 are non-blank and 137 are both non-blank and non-comment, expressing 11 run blocks and 26 assert blocks. We give both line counts because they differ by the 16 comment lines and one of them is the figure a reader would compute; quoting the larger under the stricter description would inflate the suite by 12%. Counts were obtained by line-oriented pattern matching over the suite sources, and the sources are in the replication package so the count can be reproduced or disputed.

We do not report a code-volume ratio against an external test framework. Doing so would require an independently authored equivalent suite, and no such implementation exists for this infrastructure. Constructing one ourselves would place both sides of the comparison in the hands of the party with an interest in the outcome, which is not a measurement we would ask a reader to accept. The claim that native testing reduces test-suite volume relative to an external framework is therefore outside the scope of this paper. We note only that the native suite requires no language runtime, dependency manifest, or build step beyond the Terraform binary already needed to plan the configuration — a qualitative difference in toolchain surface, not a quantitative claim about effort.

TABLE VII
Layer attribution over the vulnerable cases. Overlap is reported explicitly: layers are not assumed to detect disjoint sets. Layer 2 (native module testing) is not scored: it validates module behaviour rather than detecting resource misconfiguration, so it has no detection count on this corpus.

Configuration	Detected	Recall (%)
L1 (repository edge)	1	3.85
L3 (compiled plan)	20	76.92
overlap: L1 (repository edge) ∩ L3 (compiled plan)	0	0.00
Union of scored layers	21	80.77

D. RQ4: Layer Attribution

Table VII attributes detections to the layer that produced them. Each layer is scored by the same normalization and matching procedure applied to the external scanners, so a layer is credited only when its finding maps to the control the case seeds; no layer receives credit by construction.

The table reports pairwise intersections and the union alongside the per-layer counts, because the per-layer counts alone cannot distinguish a complementary pipeline from a redundant one. If the layers were reported additively, a case detected by every layer would be counted repeatedly and the composed pipeline would appear to achieve more than it does. The union is therefore the only figure that characterises the pipeline as a whole, and any excess of the summed per-layer counts over the union is redundancy, not additional coverage.

Two structural constraints bound what this attribution can show, and both suppress apparent complementarity rather than inflate it. The repository-edge layer detects hardcoded credentials and comparable textual patterns; it is insensitive to every control expressed as a resource attribute, so on a corpus dominated by attribute-level controls its contribution is necessarily small. The plan-level layer, conversely, can only be credited on cases for which a plan was produced; cases whose planning failed are excluded from its denominator rather than recorded as misses. The native module-testing layer validates the project's own infrastructure and is not exercised per benchmark case, so it is excluded from this table and reported under RQ3 instead. Its omission is a scope boundary, not a null result.

E. RQ5: External Generalization

Generalization is assessed only on subsets whose configurations are present in the replication package. The distinction matters here because the external manifests enumerate substantially more entries than there are configurations on disk, and a manifest entry without a configuration is a citation, not a case. Counting such entries would inflate the external corpus by roughly two orders of magnitude while contributing no measurement. Section V-A reports the declared and present counts side by side for this reason.

The external configurations that are present are drawn from published CIS Amazon Web Services Benchmark examples and carry ground-truth labels derived from the benchmark control each example illustrates; they enter the admissible corpus on the same terms as the internal cases and are validated by the same procedure.

One defect in how they were stored is worth reporting, because its signature is invisible in aggregate. A case is scanned as a directory rather than as a file, and these four configurations initially shared one directory. Every tool therefore received identical input for all four cases and returned a byte-identical finding set for each, while the scoring stage compared that one result against four different controls. Four rows entered each confusion matrix where one observation existed. This does not add noise so much as misstate precision: interval width is driven by the number of independent observations, so duplicated cases narrow every interval that includes them without contributing evidence. Nothing in the aggregate output distinguishes this from four genuinely independent cases that happen to agree. Each configuration now occupies its own directory, and the corpus loader refuses to load a corpus in which two cases share one, rather than scoring it.

We report this because it bears on how the external subset should be read, and because the same trap applies to any benchmark whose unit of analysis is a directory: case isolation is a property that has to be enforced, not assumed. Their number is small enough that a per-subset recall estimate would carry an interval spanning most of the unit interval, so we report them as part of the combined admissible corpus and treat the external subset as a check that the harness operates on configurations it did not generate, rather than as an independent effectiveness estimate. We make no claim that effectiveness measured here transfers to production infrastructure; establishing that requires a labelled corpus of real-world configurations, which to our knowledge does not exist for this domain and which we identify as the principal obstacle to progress in Section X.

VII. Discussion

Two findings from building the harness bear on how IaC security tooling should be evaluated.

First, the matching criterion dominates the headline number. Because native rule identifiers carry no shared semantics, any comparison implicitly chooses what counts as a correct detection, and that choice moves recall by more than the differences between tools typically reported in the literature. A benchmark that does not state its matching criterion is not reproducible, whatever its corpus size. This restates Habib and Pradel's finding for IaC [23].

Second, taxonomy coverage is a bias mechanism. When a canonical taxonomy is authored around one tool's rule set, findings that other tools emit under identifiers the taxonomy lacks are scored as misses. The effect is systematic and favours the authoring tool. Auditing observed rule identifiers against the taxonomy after execution,

and reporting the unmapped remainder, is the minimum control; during this work the audit identified rules from comparator tools that the initial taxonomy omitted and that would otherwise have been scored as failures.

A third observation concerns plan-level evaluation. Because the compiled plan is produced after expression resolution, policies observe concrete values and avoid a class of interpolation-related imprecision that source-level analysis must otherwise reason about [21]. The cost is that plan generation dominates the layer's latency and requires a working provider configuration, so the comparison between source-level and plan-level evaluation is a trade-off rather than a ranking.

VIII. Threats to Validity

A. Construct Validity

The controlled corpus was authored by the same investigator who implemented the evaluated policy set. Ground-truth labels therefore encode the same reading of each control that the policies encode, and agreement between the two is not independent evidence of detection capability. This is the most serious limitation of the evaluation, and it is only partly mitigated.

Two mitigations apply. Labels derive from the text of the originating control rather than from any policy implementation, and both the specifications and the generator that consumes them are released, so a reader can audit the derivation instead of taking it on trust. And generalization is assessed on configurations the investigator did not author.

A third check bears on it partially. The blind relabelling pass of Section V-C agreed with 47 of 48 recorded labels, and the one disagreement it surfaced was a genuine specification defect of exactly the predicted kind: a control whose stated text was weaker than the requirement its label encoded, where the stricter reading matched the reference policy set. That the pass found such a case is evidence the mechanism is real, not merely conceivable.

It is not, however, a sufficient mitigation, and we do not present it as one. The second rater is automated, so no claim of human inter-rater reliability is made, and its reading of control texts derives from the same public documentation the specifications were written from — so the two raters share provenance and agreement is biased upward. A single investigator still wrote every specification. The external subset is the only part of the corpus outside this failure mode entirely, and it is too small to estimate effectiveness on its own. Independent relabelling by a second human who has not read the policy set remains the correct mitigation and remains outstanding.

Results on the controlled corpus should therefore be read as an upper bound on field performance. A high score on a developer-authored corpus is at least as consistent with insufficient corpus difficulty as with strong detection, and we do not interpret it as the latter.

B. Internal Validity

Comparator tools ran with default rule sets, whereas the plan-level policies were authored with knowledge of the corpus taxonomy. This asymmetry favours the plan-level layer. It is disclosed rather than adjusted for, because equivalent tuning effort across heterogeneous engines cannot be operationalised objectively; readers should treat the comparison as bounding what default-configuration scanning achieves, not as a ranking of the tools' attainable capability.

One plan-level policy was corrected after its results had been inspected, and the correction is disclosed here because it is exactly the iteration the comparators did not receive. The audit-log-encryption rule tested for the presence of a key-identifier attribute in the plan document. Terraform serialises three configuration states into that document: an attribute set to a literal carries its value; an attribute set from a reference to another resource is omitted and recorded as not-yet-known; and an attribute that is not set at all is present with a null value. A presence test therefore fails on the configuration that omits the attribute and succeeds on the configuration that sets it from a reference, which inverts the control rather than weakening it. The compliant case was reported as violating and the violating case as compliant, and the two errors concealed each other in the aggregate: the confusion matrix showed one false positive and one false negative, a signature indistinguishable from ordinary imprecision.

The corrected rule distinguishes not-set from not-yet-known and reports only the former. The latter is genuinely indeterminate at plan time — the attribute is set, but nothing in the plan establishes what to — and is reported as neither compliant nor violating. This is a property of plan-level evaluation and not of this implementation: any policy engine reading plan documents inherits the same three-way distinction, and any that collapses it will invert the same class of control. We report it as a finding about the evaluation layer rather than as a defect that has been repaired, and note that the same defect class does not affect block-typed attributes, whose unconfigured form is absent from the plan and can be tested for presence safely.

Because this correction was informed by inspecting corpus results, the plan-level figures should be read as reflecting one round of iteration that Checkov and tfsec did not receive. The pre-correction and post-correction results are both in the replication package.

Latency was measured on a single host with recorded specifications. Absolute figures do not transfer across environments. Latency additionally measures each tool at its own interface: the plan-level figure covers policy evaluation over an already-compiled plan document and excludes the terraform init and terraform plan invocations that produce it, which dominate that layer's true wall-clock cost in continuous integration. The source-level scanners have no comparable preparation step. Plan-level and source-level latencies are therefore not interchangeable, and the plan-level figure is a lower bound on deployed cost.

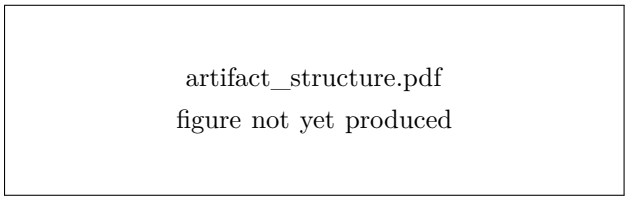

Fig. 3. Replication package layout.

C. External Validity

The operational deployment covers a single project maintained by a single developer. Observations concerning developer friction, escalation workflow, and policy ownership are experience-report evidence and do not generalise to multi-team organisations, other cloud providers, or other IaC technologies.

D. Scope

The corpus is deterministic rather than adversarial. Reported effectiveness characterises performance on a published taxonomy of known misconfiguration classes. No claim is made regarding novel or deliberately obfuscated insecure configurations, or an adversary who has read the policy set.

IX. Replication Package

The replication package contains the benchmark corpus with ground-truth annotations and the generator specifications from which those annotations derive; the canonical control taxonomy as auditable data; the per-case output of the blind relabelling pass of Section V-C, with its reasons and its stated limitations; the execution harness, which records raw tool output, resolved versions, and repeated latency samples; the normalization engine; the statistical implementation, with a test suite pinning each method to independently derived values; and the analysis stage that generates every table in this paper. It also retains the measured results from before the plan-level policy correction disclosed in Section VIII-B, so that the disclosure can be checked rather than taken on trust.

Two properties are worth stating explicitly. The harness declines to emit results when no case is admissible, rather than producing an empty but plausible table. And the statistical implementation refuses to report a proportion whose denominator is zero, returning a not-estimable marker instead of a value a reader could mistake for a measurement.

https://github.com/mchittineni/IaCSecBench
https://doi.org/10.5281/zenodo.21645016

X. Conclusion

This paper contributes an evaluation methodology for IaC security validation tools: a finding-normalization scheme that makes heterogeneous scanner output commensurable, an explicit treatment of matching strictness, a mechanically enforced corpus admissibility procedure, and a statistical protocol suited to paired small-sample comparison. The accompanying framework composes repository-edge validation, native module testing, and plan-level policy evaluation, and is released with a harness in which every reported figure is a generated artefact traceable to recorded tool output.

The methodological findings are the transferable result. Matching criteria and taxonomy coverage exert more influence on reported detection rates than the differences between tools that such reports typically claim to establish, which suggests that comparative claims in this area should be accompanied by both, and that benchmark size is a weaker indicator of evidential strength than admissibility and interval width.

Future work should extend the taxonomy to IaC technologies beyond Terraform, enlarge the compliant-baseline portion of the corpus so that false-positive behaviour becomes estimable, and evaluate whether plan-level policy evaluation retains its advantage on configurations using deeply nested modules.

Acknowledgment

The author thanks the maintainers of Terraform, Open Policy Agent, Checkov, and tfsec. AI-assisted tooling was used for manuscript copy-editing; all technical content, measurements, and conclusions are the author's.

References

[1] K. Morris, Infrastructure as Code: Dynamic Systems for the Cloud Age, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2020.
[2] M. Guerriero, M. Garriga, D. A. Tamburri, and F. Palomba, "Adoption, support, and challenges of infrastructure-as-code: Insights from industry," in Proc. IEEE Int. Conf. Software Maintenance and Evolution (ICSME), 2019, pp. 580–589.
[3] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. Katz, A. Konwinski, G. Lee, D. Patterson, A. Rabkin, I. Stoica, and M. Zaharia, "A view of cloud computing," Communications of the ACM, vol. 53, no. 4, pp. 50–58, 2010.
[4] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in Proc. 41st Int. Conf. Software Engineering (ICSE), Montreal, QC, Canada, 2019, pp. 164–175.
[5] A. Rahman, M. R. Rahman, C. Parnin, and L. Williams, "Security smells in Ansible and Chef scripts: A replication study," ACM Trans. Software Engineering and Methodology, vol. 30, no. 1, pp. 1–31, 2021.
[6] A. Verdet, M. Hamdaqa, L. Da Silva, and F. Khomh, "Exploring security practices in infrastructure as code: An empirical study," Empirical Software Engineering, vol. 29, no. 4, 2024.
[7] L. Bass, I. Weber, and L. Zhu, DevOps: A Software Architect's Perspective. Boston, MA, USA: Addison-Wesley, 2015.
[8] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Boston, MA, USA: Addison-Wesley, 2010.
[9] C. J. Clopper and E. S. Pearson, "The use of confidence or fiducial limits illustrated in the case of the binomial," Biometrika, vol. 26, no. 4, pp. 404–413, 1934.
[10] A. Agresti and B. A. Coull, "Approximate is better than "exact" for interval estimation of binomial proportions," The American Statistician, vol. 52, no. 2, pp. 119–126, 1998.

[11] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.

[12] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," Neural Computation, vol. 10, no. 7, pp. 1895–1923, 1998.

[13] S. Holm, "A simple sequentially rejective multiple test procedure," Scandinavian Journal of Statistics, vol. 6, no. 2, pp. 65–70, 1979.

[14] A. Rahman, E. Farhana, C. Parnin, and L. Williams, "Gang of eight: A defect taxonomy for infrastructure as code scripts," in Proc. 42nd Int. Conf. Software Engineering (ICSE), 2020, pp. 752–764.

[15] A. Rahman, S. I. Shamim, D. B. Bose, and R. Pandita, "Security misconfigurations in open source Kubernetes manifests: An empirical study," ACM Trans. Software Engineering and Methodology, vol. 32, no. 4, pp. 1–36, 2023.

[16] M. S. I. Shamim, F. A. Bhuiyan, and A. Rahman, "XI commandments of Kubernetes security: A systematization of knowledge related to Kubernetes security practices," in Proc. IEEE Secure Development Conf. (SecDev), 2020, pp. 58–64.

[17] S. Dalla Palma, D. Di Nucci, and D. A. Tamburri, "Toward a catalog of software quality metrics for infrastructure code," Journal of Systems and Software, vol. 170, p. 110726, 2020.

[18] S. Dalla Palma, D. Di Nucci, F. Palomba, and D. A. Tamburri, "Within-project defect prediction of infrastructure-as-code using product and process metrics," IEEE Trans. Software Engineering, vol. 48, no. 6, pp. 2086–2104, 2022.

[19] M. Chiari, M. De Pascalis, and M. Pradella, "Static analysis of infrastructure as code: A survey," in Proc. IEEE 19th Int. Conf. Software Architecture Companion (ICSA-C), 2022, pp. 218–225.

[20] N. Saavedra and J. F. Ferreira, "GLITCH: Automated polyglot security smell detection in infrastructure as code," in Proc. 37th IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2022, pp. 1–12.

[21] R. Opdebeeck, A. Zerouali, and C. De Roover, "Control and data flow in security smell detection for infrastructure as code: Is it worth the effort?" in Proc. 19th Int. Conf. Mining Software Repositories (MSR), 2022, pp. 534–545.

[22] R. Opdebeeck, A. Zerouali, C. Velázquez-Rodríguez, and C. De Roover, "Smelly variables in Ansible infrastructure code: Detection, prevalence, and lifetime," in Proc. 18th Int. Conf. Mining Software Repositories (MSR), 2021, pp. 61–72.

[23] A. Habib and M. Pradel, "How many of all bugs do we find? a study of static bug detectors," in Proc. 33rd IEEE/ACM Int. Conf. Automated Software Engineering (ASE), 2018, pp. 317–328.

[24] OWASP Foundation, "OWASP benchmark project," https://owasp.org/www-project-benchmark/, 2023, accessed Aug. 2, 2026.

[25] Open Policy Agent, "Open Policy Agent documentation," https://www.openpolicyagent.org/docs, Cloud Native Computing Foundation, 2024, accessed Aug. 2, 2026.

[26] HashiCorp, "Terraform tests," https://developer.hashicorp.com/terraform/language/tests, 2024, accessed Aug. 2, 2026.

[27] Center for Internet Security, "CIS Amazon Web Services foundations benchmark, v3.0.0," https://www.cisecurity.org/benchmark/amazon_web_services, 2024, accessed Aug. 2, 2026.

[28] National Institute of Standards and Technology, "Security and privacy controls for information systems and organizations," NIST, Tech. Rep. Special Publication 800-53, Rev. 5, 2020.

[29] A. Shostack, Threat Modeling: Designing for Security. Indianapolis, IN, USA: John Wiley & Sons, 2014.

[30] B. W. Matthews, "Comparison of the predicted and observed secondary structure of T4 phage lysozyme," Biochimica et Biophysica Acta (BBA) – Protein Structure, vol. 405, no. 2, pp. 442–451, 1975.

[31] A. Arcuri and L. Briand, "A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," Software Testing, Verification and Reliability, vol. 24, no. 3, pp. 219–250, 2014.

Manideep Chittineni is a Senior Platform, Cloud and DevOps Engineer based in the United Kingdom. His work concerns cloud platform automation, infrastructure security validation, and continuous delivery engineering.